



MODULE 0 – PRÉREQUIS & INSTALLATION DES OUTILS

Formation Flutter Professionnelle – 50 heures

Session préparatoire (à réaliser avant le Jour 1)

Durée totale conseillée : 3 heures (en présentiel) ou à réaliser en auto-formation avant le Jour 1.

SUPPORT DE COURS – Théorie complète

1. Prérequis de la formation

Prérequis techniques obligatoires

Avant de commencer, vous devez maîtriser les concepts suivants.

Si ce n'est pas le cas, complétez les ressources recommandées **avant** le Module 1.

Compétence	Niveau requis	Auto-évaluation
Variables et types	Je sais déclarer des variables, comprendre les types (String, int, bool)	
Conditions	Je maîtrise if/else, switch, les opérateurs logiques (&&, \ \ , !)	
Boucles	Je sais utiliser for, while, forEach	
Fonctions	Je peux créer des fonctions avec paramètres et valeurs de retour	
Tableaux/Listes	Je manipule des collections : ajout, suppression, parcours, filtres	
Programmation Orientée Objet	Je comprends les classes, objets, constructeurs, héritage	
Notions de base réseau	Je sais ce qu'est une API, une requête HTTP, un JSON	
Ligne de commande	Je suis à l'aise avec le terminal (cd, ls, mkdir, commandes de base)	
Git (bases)	Je connais git init, add, commit, push, pull	

Ressources de remise à niveau (si besoin)

- **Algorithmique** : <https://www.codecademy.com/learn/learn-how-to-code> (gratuit, 6h)
- **POO** : <https://www.sololearn.com/fr/learn/courses/java-intermediate> (modules POO, gratuit)
- **Git** : <https://www.atlassian.com/fr/git/tutorials> (30 min)
- **Ligne de commande** : <https://cmdchallenge.com> (entraînement ludique)



2. Comprendre l'écosystème Flutter

Qu'est-ce que Flutter ? Flutter est un **SDK (Software Development Kit)** open-source développé par Google. Il permet de créer des applications :

- **Mobile** : iOS et Android (natif)
- **Web** : application web progressive
- **Desktop** : Windows, macOS, Linux

Une seule base de code pour toutes les plateformes.

Flutter vs les autres approches

Approche	Technologie	Avantage	Inconvénient
Natif pur	Swift (iOS) / Kotlin (Android)	Performance maximale, accès 100% natif	2 codebases, 2 équipes, coût ×2
Hybride WebView	Ionic, Cordova	Web standard, rapide à prototyper	Performance médiocre, feeling non natif

Approche	Technologie	Avantage	Inconvénient
Bridge natif	React Native	Vrais composants natifs, communauté large	Bridge JavaScript/Natif = latence, bugs
Rendu propriétaire	Flutter	Performance native, UI identique partout, hot reload	Taille APK plus grande, courbe d'apprentissage

Pourquoi choisir Flutter aujourd'hui ?

- **Performance** : 60/120 FPS grâce au moteur de rendu Skia/Impeller
- **Productivité** : Hot reload en < 1 seconde
- **UI cohérente** : même design sur iOS et Android (ou adaptatif si souhaité)
- **Écosystème riche** : 25 000+ packages sur pub.dev
- **Marché du travail** : demande forte et croissante

Dart, le langage de Flutter

Dart est le langage de programmation associé à Flutter. Il est :

- **Facile à apprendre** : syntaxe proche de Java, C#, JavaScript
- **Compilé** : performances natives en production (AOT)
- **Interprété** : hot reload ultra-rapide en développement (JIT)
- **Null-safe** : sécurité intégrée contre les erreurs null (depuis 2021)



3. Installation complète de l'environnement

Configuration matérielle minimale

Composant	Minimum	Recommandé
RAM	8 Go	16 Go
Disque	10 Go libres	20 Go libres (SSD fortement conseillé)
Processeur	Intel i3 / AMD Ryzen 3	Intel i5 / AMD Ryzen 5
Système	Windows 10, macOS 10.14, Linux 64-bit	Windows 11, macOS 13+, Ubuntu 22.04

Étape 1 – Télécharger Flutter SDK Windows

```
# Via Git
git clone https://github.com/flutter/flutter.git -b stable

# Ou téléchargez le zip depuis :
# https://docs.flutter.dev/get-started/install/windows
```

macOS

```
# Via Git
git clone https://github.com/flutter/flutter.git -b stable

# Ou via Homebrew (non officiel)
brew install --cask flutter
```

Linux

```
# Via Git
git clone https://github.com/flutter/flutter.git -b stable

# Ou via snap
sudo snap install flutter --classic
```

Étape 2 – Configurer le PATH Windows (PowerShell en administrateur)

```
# Remplacez C:^^Flutter par votre chemin réel

[Environment]::SetEnvironmentVariable(
    "Path",
    $env:Path + ";C:\flutter\bin",
    "User"
)
```

Puis **redémarrez votre terminal**. macOS /

Linux

```
# Ouvrez ~/.zshrc (macOS récent) ou ~/.bashrc (Linux)
```



```
# Ajoutez cette ligne à la fin :  
export PATH="$PATH:/chemin/vers/flutter/bin"  
  
# Rechargez la configuration  
source ~/.zshrc
```

Étape 3 – Vérifier l’installation avec flutter doctor

```
flutter doctor
```

Sortie idéale :

```
Doctor summary (to see all details, run flutter doctor -v):  
[ ] Flutter (Channel stable, 3.24.0, ...)  
[ ] Android toolchain - develop for Android devices (Android SDK 34.0.0)  
[ ] Chrome - develop for the web  
[ ] Android Studio (version 2023.1)  
[ ] VS Code (version 1.85)  
[ ] Connected device (2 available)  
[ ] Network resources
```

Problèmes courants et solutions

Problème	Symptôme	Solution
Android SDK manquant	[] Android toolchain	Installer Android Studio → SDK Manager → cocher “Android SDK”
cmdline-tools manquant	cmdline-tools component is missing	SDK Manager → SDK Tools → “Android SDK Command-line Tools (latest)”
Licence non acceptée	Some Android licenses...	Exécuter flutter doctor --android-licenses puis taper y
Aucun appareil	[] Connected device	Brancher un téléphone (mode développeur) ou créer un émulateur
VS Code non détecté	[] VS Code	Installer VS Code + extension Flutter

Étape 4 – Installer Android Studio (recommandé même avec VS Code)

1. Téléchargez sur <https://developer.android.com/studio>
2. Lancez l’installation standard
3. Ouvrez Android Studio → **SDK Manager** (icône en haut à droite)
4. Onglet **SDK Platforms** : cochez au moins :
 - Android 14.0 (API 34)
 - Android 13.0 (API 33)
5. Onglet **SDK Tools** : cochez :
 - Android SDK Build-Tools
 - Android SDK Command-line Tools (latest)



- Android Emulator
- Android SDK Platform-Tools

6. Cliquez **Apply** puis attendez le téléchargement

Étape 5 – Créer un émulateur Android

1. Android Studio → **Virtual Device Manager** (icône téléphone en haut)
2. Cliquez **Create Device**
3. Choisissez un modèle (recommandé : **Pixel 7**)
4. Sélectionnez une image système : **Tiramisu (API 33)** ou **UpsideDownCake (API 34)**
5. Nommez votre émulateur et cliquez **Finish**
6. Lancez-le avec le bouton 

Astuce performance : dans les paramètres de l'émulateur, activez :

- **Graphics** : Hardware – GLES 2.0
- **Boot option** : Cold boot (plus stable pour les tests)

Étape 6 – Configurer VS Code (alternative légère)

1. Installez VS Code : <https://code.visualstudio.com>
2. Extensions indispensables (Ctrl+Shift+X) :
 - **Flutter** (Dart Code) – extension officielle
 - **Dart** – support du langage
 - **Awesome Flutter Snippets** – raccourcis de code
 - **Flutter Tree** – visualisation de l'arbre des widgets
 - **Error Lens** – affichage des erreurs en ligne
3. Configuration recommandée (Ctrl+, puis recherchez) :
 - editor.formatOnSave : **true**
 - editor.rulers : ajoutez **80** (limite de longueur de ligne)
 - dart.previewFlutterUiGuides : **true** (lignes de guidage)

4. Premier projet et hot reload

Créer un projet

```
flutter create mon_premier_projet  
cd mon_premier_projet
```

Structure générée :

```
mon_premier_projet/  
├── android/           # Code natif Android (Kotlin)  
├── ios/               # Code natif iOS (Swift)  
├── lib/  
│   └── main.dart      # Point d'entrée de l'application  
├── test/              # Tests (vide au départ)  
├── pubspec.yaml       # Dépendances et configuration  
└── analysis_options.yaml
```



Lancer l'application

```
# Sur l'émulateur ou appareil connecté  
flutter run  
  
# Sur un appareil spécifique (si plusieurs connectés)  
flutter run -d <device_id>  
  
# En mode release (performance réelle)  
flutter run --release
```

Découvrir le hot reload

1. Lancez l'application avec flutter run
2. Ouvrez lib/main.dart
3. Modifiez le texte 'Hello World' en 'Hello Flutter !'
4. Sauvegardez le fichier (Ctrl+S)
5. Observez : l'application se met à jour **en moins d'une seconde** sans perdre l'état !

Commandes utiles dans le terminal de flutter run :

Touche	Action
r	Hot reload (met à jour l'UI, garde l'état)
R	Hot restart (redémarre l'app, perd l'état)
h	Aide
q	Quitter
p	Basculer l'affichage des performances (overlay)
t	Basculer le mode widget inspector

5. Git et GitHub – Préparation du travail collaboratif

Pourquoi Git dès le début ? En entreprise, tout projet Flutter est versionné. Dès le Module 2, vous pousserez votre code sur GitHub. Il est donc essentiel d'avoir un compte et un repository prêt.

Créer un compte GitHub

1. Rendez-vous sur <https://github.com>
2. Créez un compte avec votre email professionnel
3. Vérifiez votre email

Configurer Git localement

```
# Identité  
git config --global user.name "Votre Nom"  
git config --global user.email "votre@email.com"  
  
# Éditeur par défaut (VS Code)
```



```
git config --global core.editor "code --wait"
```

```
# Vérification
```

```
git config --list
```

Créer un repository et pousser le projet

```
# Dans le dossier de votre projet Flutter
```

```
cd mon_premier_projet
```

```
# Initialiser Git
```

```
git init
```

```
# Créer un fichier .gitignore (Flutter en a déjà un, vérifiez qu'il existe)
```

```
# Ignorer les fichiers générés
```

```
cat > .gitignore << EOF
```

```
# Flutter
```

```
.dart_tool/
```

```
.packages
```

```
build/
```

```
*.lock
```

```
# IDE
```

```
.idea/
```

```
.vscode/
```

```
# OS
```

```
.DS_Store
```

```
Thumbs.db
```

```
EOF
```

```
# Premier commit
```

```
git add .
```

```
git commit -m "Initial commit : projet Flutter de base"
```

```
# Connexion à GitHub (créez d'abord un repo vide sur GitHub)
```

```
git remote add origin https://github.com/VOTRE_USER/mon_projet.git
```

```
git branch -M main
```

```
git push -u origin main
```

Pour la formation : créez un repository nommé formation-flutter-fil-rouge et poussez-y votre projet initial. Vous y pousserez chaque module.

TRAVAUX PRATIQUES – Module 0

TP1 – Vérification de l'environnement (auto-évaluation)

Objectif S'assurer que tout est installé et fonctionnel avant le Jour 1.



Checklist à cocher Flutter SDK

- La commande flutter --version affiche une version (ex. 3.24.0)
- La commande flutter doctor affiche **toutes les cases vertes** []
- Si une case est [] ou [!], la résoudre avant de continuer

Émulateur / Appareil

- Un émulateur Android est créé et peut se lancer
- OU un téléphone physique est branché en mode développeur
- La commande flutter devices affiche au moins un appareil

IDE

- VS Code ou Android Studio est installé
- L'extension Flutter est installée et active
- La coloration syntaxique Dart fonctionne

Premier projet

- flutter create test_env fonctionne sans erreur
- cd test_env && flutter run lance l'application
- Le compteur s'incrémente quand on clique le bouton +
- Le hot reload fonctionne (modifiez le texte, sauvegardez, observez)

Git

- Un compte GitHub est créé
- git --version fonctionne dans le terminal
- Un repository est créé sur GitHub
- Le projet Flutter a été poussé sur ce repository

Livrable Capture d'écran de flutter doctor avec toutes les cases vertes, envoyée au formateur.

TP2 – Exploration du projet généré

Objectif Comprendre la structure d'un projet Flutter avant de coder.

Consignes

1. Créez un projet flutter create exploration
2. Ouvrez-le dans VS Code / Android Studio
3. Ouvrez lib/main.dart et lisez le code en entier
4. Identifiez et notez (dans un fichier NOTES.md) :
 - Le widget racine
 - Le widget qui gère l'état (compteur)



- Où est défini le thème
- Quelle méthode déclenche la mise à jour de l'interface

5. Modifiez progressivement :

- Changez la couleur du thème (seedColor)
- Changez le texte du bouton
- Changez le titre de l'AppBar
- Ajoutez un deuxième bouton qui décrément le compteur

6. Pour chaque modification, utilisez le **hot reload** et observez

Questions de réflexion

- Pourquoi setState() est-il nécessaire ?
- Que se passe-t-il si vous retirez setState() ?
- Pourquoi le widget racine est-il un StatelessWidget et le compteur un StatefulWidget ?

Livrable Fichier NOTES.md avec vos observations + code modifié main.dart.

FICHE RÉCAP STAGIAIRE – Module 0

Points clés à retenir

Élément	Commande / Vérification
Version Flutter	flutter --version
Élément	Commande / Vérification
Diagnostic complet	flutter doctor
Créer un projet	flutter create nom_projet
Lancer	flutter run
Hot reload	Sauvegarder le fichier ou appuyer r
Hot restart	Appuyer R
Build release	flutter build apk (Android)
Devices disponibles	flutter devices

Installation – Ordre recommandé

1. Vérifier la configuration matérielle (RAM, disque, OS)
2. Télécharger Flutter SDK (git clone ou zip)
3. Ajouter flutter/bin au PATH
4. Vérifier avec flutter doctor
5. Installer Android Studio + SDK + cmdline-tools
6. Accepter les licences Android (flutter doctor --android-licenses)
7. Créer un émulateur Android
8. Installer VS Code + extensions Flutter/Dart
9. Créer un premier projet (flutter create)
10. Lancer et tester le hot reload
11. Créer un compte GitHub + repo + push initial



Erreurs fréquentes à éviter

1. **Oublier le PATH** → flutter n'est pas reconnu comme commande
2. **Ne pas redémarrer le terminal** après modification du PATH → commande non trouvée
3. **Ignorer les licences Android** → flutter doctor reste en [!]
4. **Lancer l'émulateur sans Intel HAXM / Hyper-V** → émulateur extrêmement lent
5. **Utiliser un disque HDD** → compilation très lente. Un SSD est quasi-obligatoire.
6. **Oublier git init avant le premier commit** → erreur "not a git repository"

Check mentale avant le Jour 1

- ☐ flutter doctor affiche uniquement des []
- ☐ Un émulateur ou téléphone est prêt
- ☐ VS Code (ou Android Studio) est configuré
- ☐ J'ai testé le hot reload avec succès
- ☐ Mon compte GitHub est créé
- ☐ Mon repo de formation est initialisé et poussé
- ☐ J'ai lu le code du projet par défaut et je le comprends
- ☐ Je maîtrise les prérequis algorithmiques/POO

QUIZ DE VALIDATION – Module 0

Durée : 10 minutes – 6 questions

Question 1 (QCM)

Quelle commande permet de vérifier que tout l'environnement Flutter est correctement installé ?

- A. flutter check
- B. flutter verify
- C. flutter doctor
- D. flutter status

Question 2 (QCM)

Quel IDE est officiellement recommandé par l'équipe Flutter ?

- A. IntelliJ IDEA uniquement
- B. Android Studio ou VS Code
- C. Xcode uniquement
- D. Sublime Text

Question 3 (QCM)

Le hot reload permet de :

- A. Mettre à jour l'application sans perdre l'état actuel
- B. Compiler une version de production



- C. Redémarrer l'application depuis zéro
- D. Installer l'application sur un nouvel appareil

Question 4 (QCM)

Quel fichier contient la configuration des dépendances et des assets d'un projet Flutter ?

- A. main.dart
- B. pubspec.yaml
- C. build.gradle
- D. AndroidManifest.xml

Question 5 (Vrai/Faux)

Il est possible de développer des applications iOS depuis un ordinateur Windows.

- A. Vrai
- B. Faux (macOS est obligatoire pour compiler iOS)

Question 6 (Ouverte)

Listez les 3 étapes minimales pour résoudre un [] Android toolchain dans flutter doctor.

Réponse attendue :

1. Installer Android Studio et le Android SDK
2. Installer les "Android SDK Command-line Tools (latest)"
3. Exécuter flutter doctor --android-licenses et accepter les licences

Barème indicatif

- 6/6 : Environnement prêt, vous pouvez commencer le Module 1
- 4-5/6 : Vérifiez les points manquants avant le Jour 1
- < 4/6 : Refaites le Module 0 en entier. Le Jour 1 sera trop difficile sinon.

ANNEXE – Ressources et liens

- **Installation Flutter officielle** : <https://docs.flutter.dev/get-started/install>
- **Flutter doctor troubleshooting** : <https://docs.flutter.dev/get-started/install/help>
- **Android Studio** : <https://developer.android.com/studio>
- **VS Code** : <https://code.visualstudio.com>
- **Git** : <https://git-scm.com/downloads>
- **GitHub** : <https://github.com>
- **pub.dev** (packages Flutter) : <https://pub.dev>
- **Communauté Discord Flutter FR** : <https://discord.gg/flutter>



CHECKLIST FINALE – À remettre au formateur avant le Jour 1

12

Stagiaire : _____ **Date :** _____

- flutter doctor affiche 100% de cases vertes (capture d'écran jointe)
- Un émulateur Android fonctionne ou un téléphone est configuré
- Le projet flutter create test s'exécute et affiche le compteur
- Le hot reload a été testé avec succès
- VS Code (ou Android Studio) est configuré avec l'extension Flutter
- Un compte GitHub est créé
- Un repository formation-flutter-fil-rouge est initialisé
- Les prérequis algorithmiques/POO sont maîtrisés (auto-évaluation)

Signature stagiaire : _____ **Signature formateur :** _____